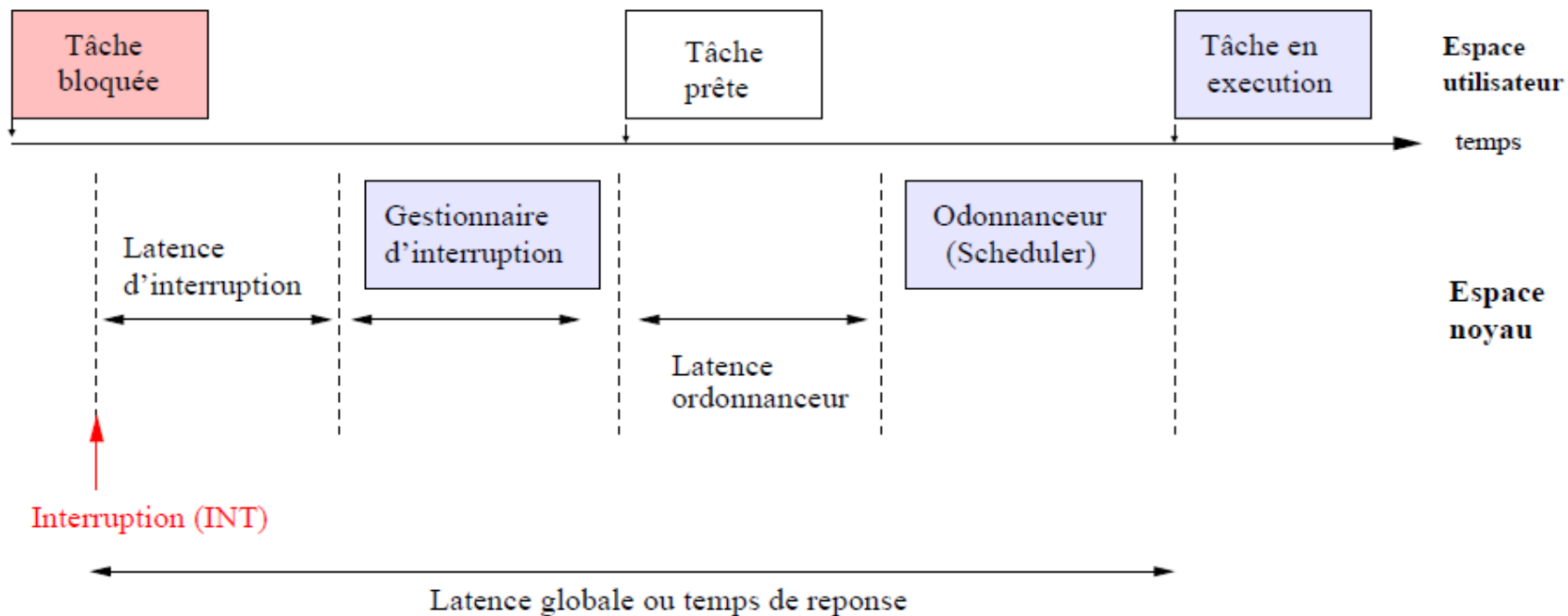


Partie 2: Systèmes Temps Réel/Xenomai

GPOS/latence globale

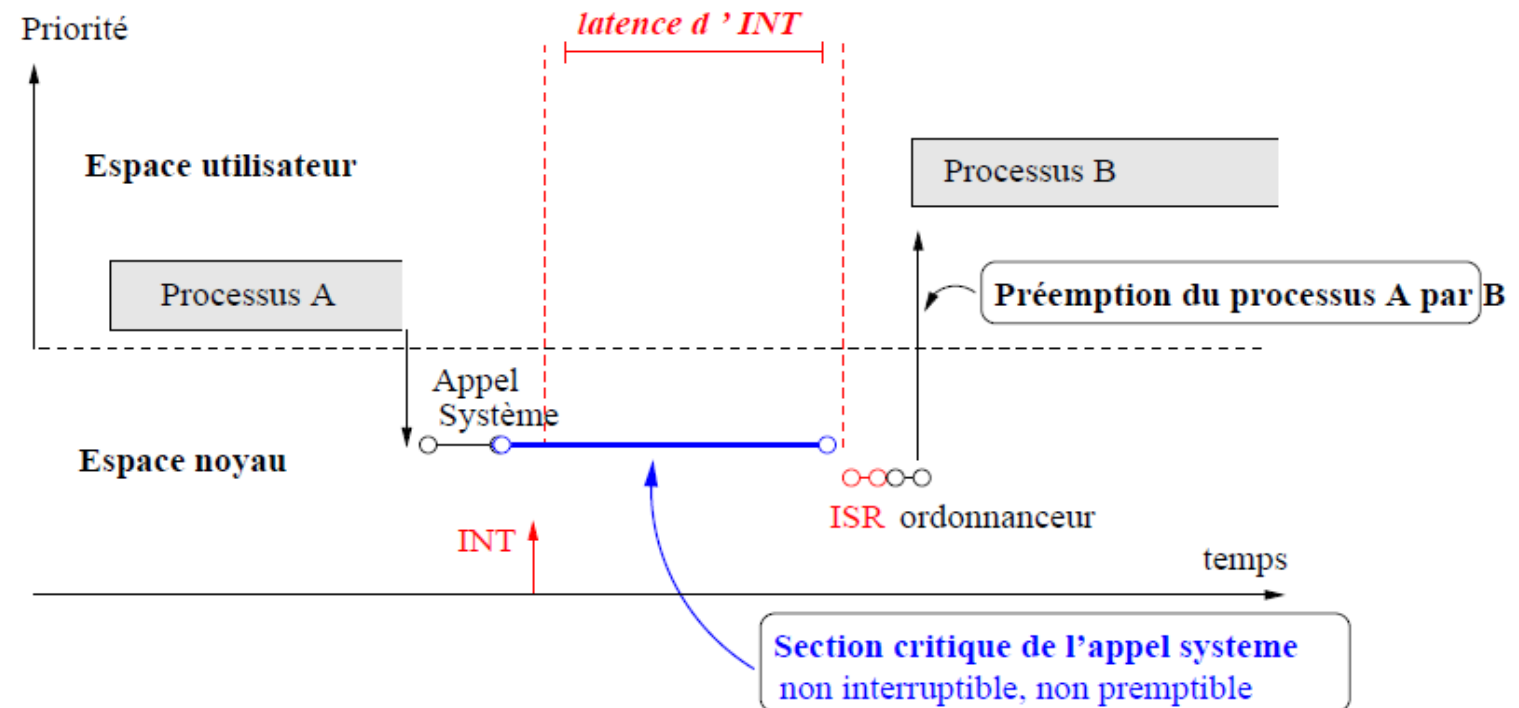
□ Répondre à une interruption

Temps de réponse (latence) = latence d'interruption + durée du gestionnaire (ISR) + latence d'ordonnancement + durée de l'ordonnanceur



Préemption/Latence du noyau linux

Réveil d'une tâche de haute priorité B par une interruption → latence importante

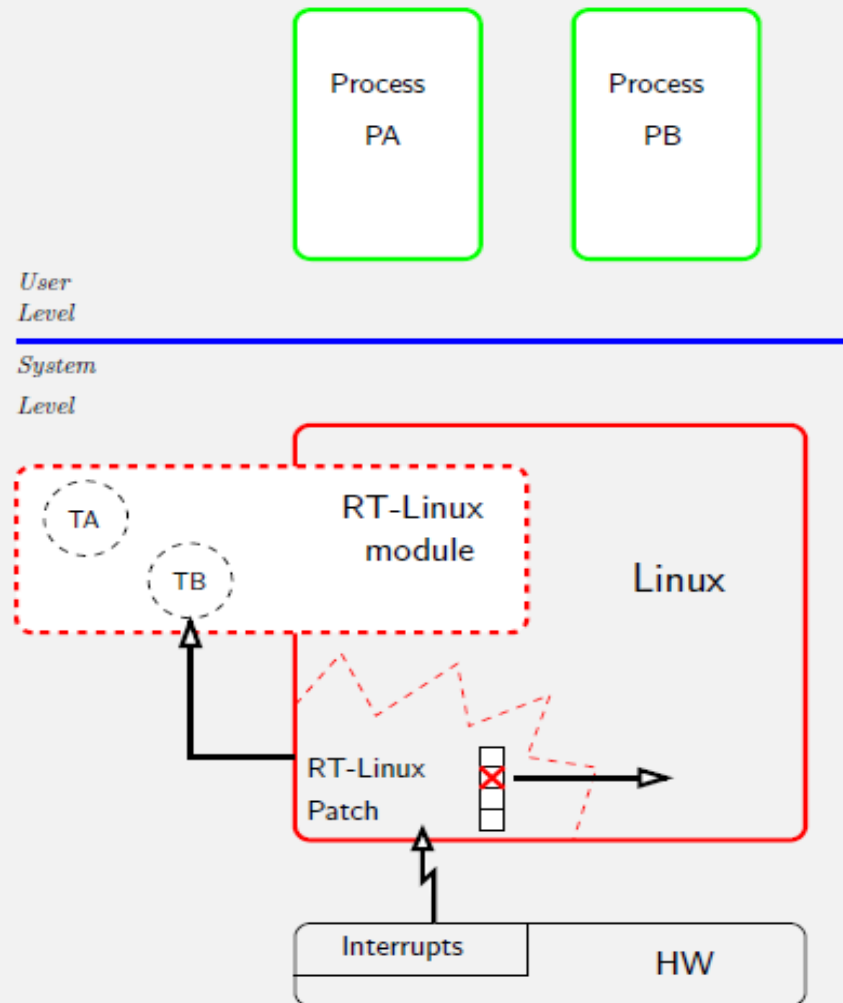


Le problème du noyau Linux est que, par défaut, il n'y a pas de préemption à l'intérieur du noyau

Solution Temps réel/RT_Linux

- ❑ RT-Linux was the first approach to making Linux more real-time
 - It consist of a patch to Linux, plus a dynamic loadable kernel module
- ❑ The patch:
 - Modifies the interrupt handling sub-system of Linux, in order to intercept and service the real-time interrupts
- ❑ The module contains:
 - the nano-kernel (scheduler + interrupt handler + libraries)
 - application code

Solution Temps réel/RT_Linux



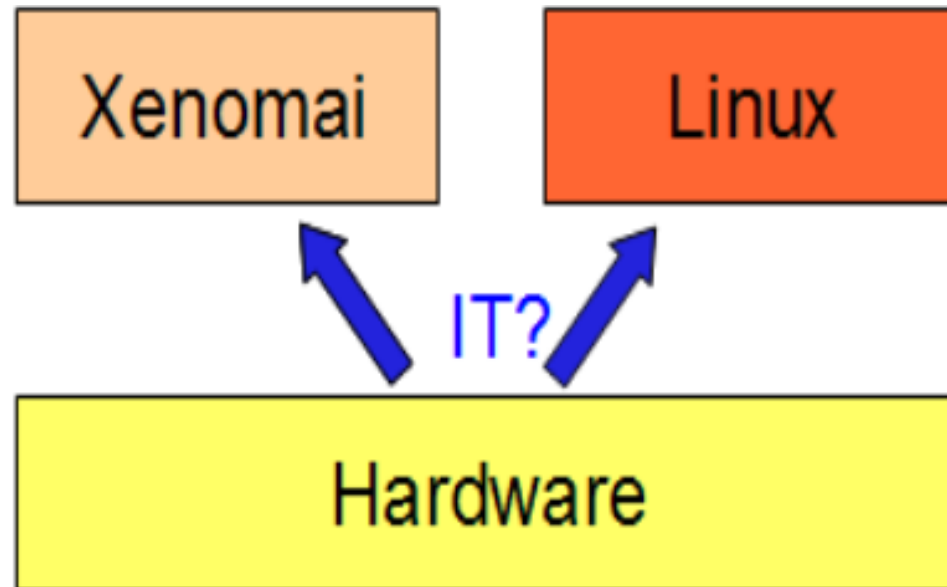
- When an interrupt is raised
 - If it is for the RT sub-system, it is redirected to the nano-kernel and to the RT-application
 - If it is for Linux, it is simply *marked* as pending
 - and it will be served only when all RT tasks have completed execution
 - Linux must be executed with interrupts always active

Solution Temps réel/RT_Linux

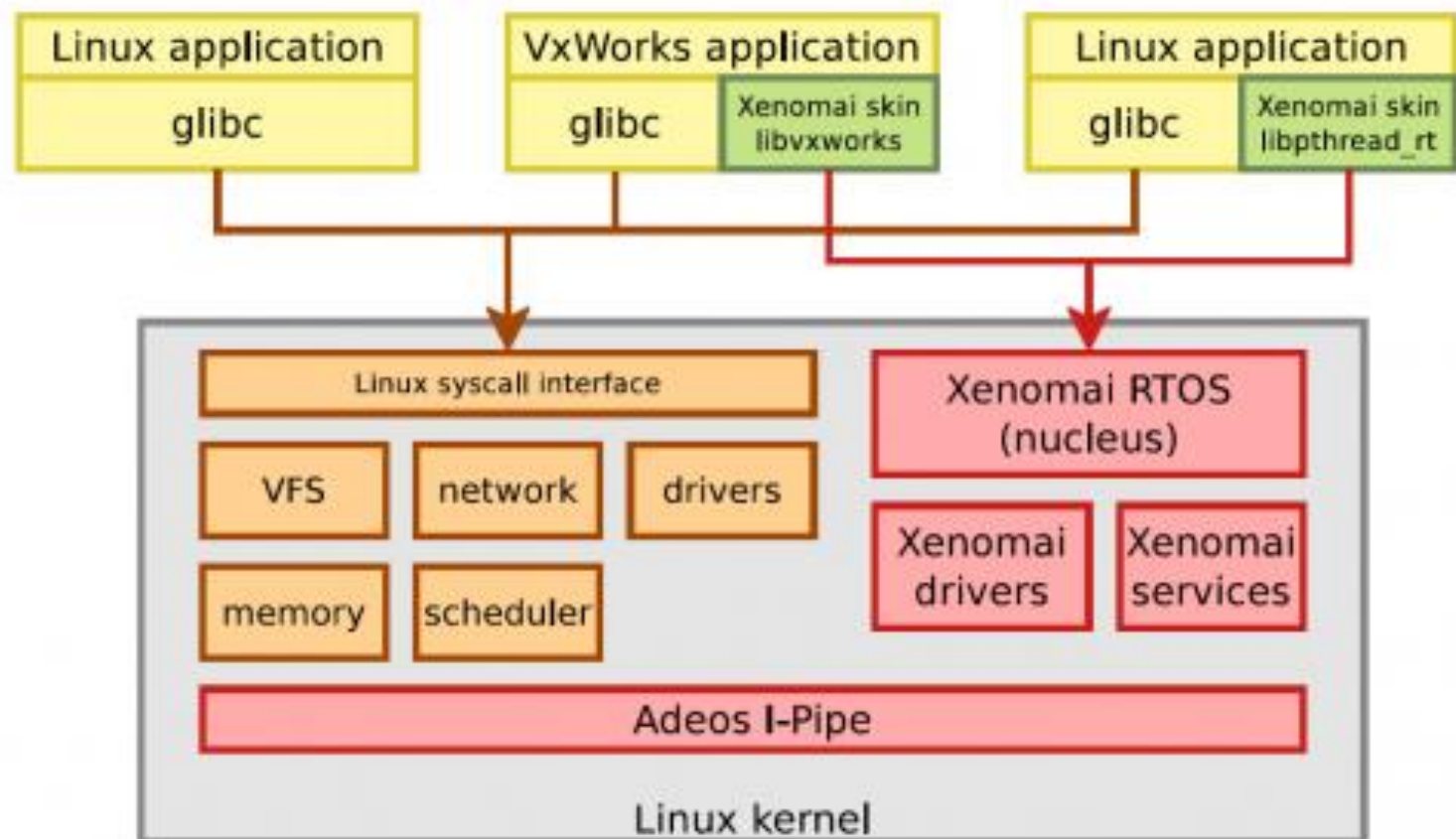
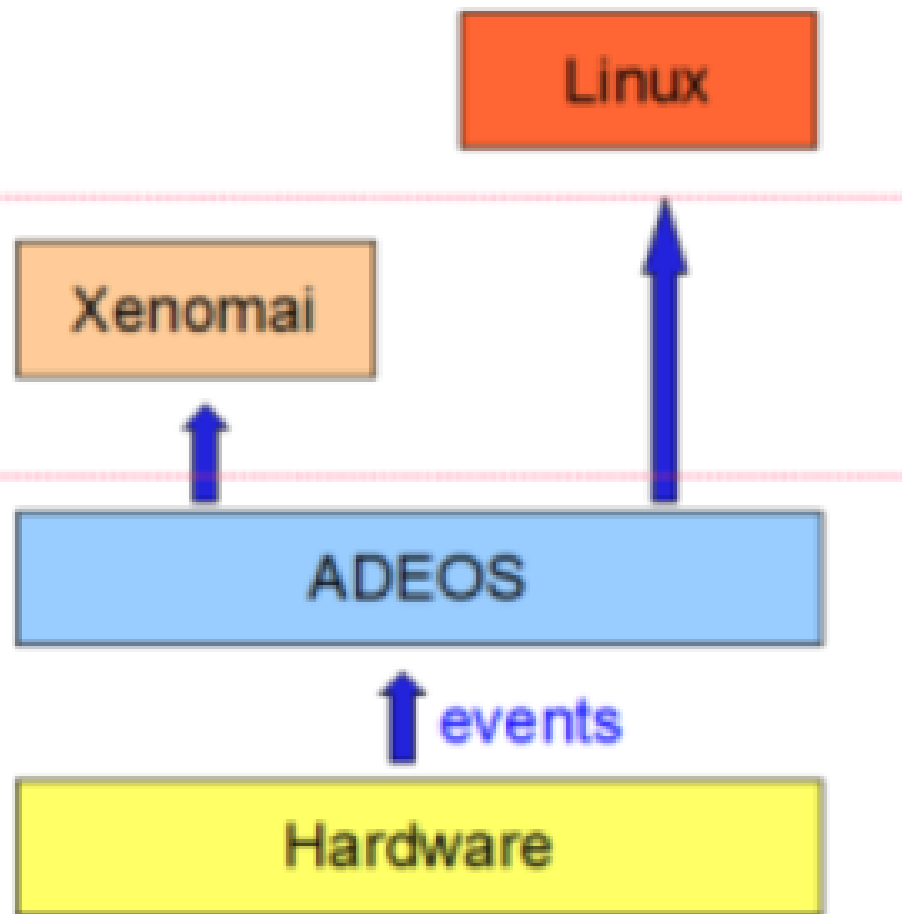
Advantages – disadvantages

- This approach has one big advantage
 - Minimum latency: RT interrupts cannot be blocked by Linux, but only by the RT sub-system
- But also some disadvantages:
 - Real-time applications run in kernel space →, so they can overwrite the kernel memory and crash the system (like any other single memory RTOS)
 - Communication between RT subsystem and Linux can only be non real-time;
 - Linux is scheduled in background → Linux tasks can starve or experience high delay
 - **It is not possible to use Linux device drivers for RT**
 - RT developers have to re-write the related device drivers on top of the RT sub-system

Solution RTOS/Xenomai



RTOS/Xenomai & Noyau Linux



ADEOS (*Adaptative Domain Environnement for Operating System*)

RTOS/Xenomai & Noyau Linux

Xenomai est un système d'exploitation temps réel qui a Linux pour tâche de fond. Linux est alors préempté comme une simple tâche. Les tâches gérées par Xenomai apportent ainsi une garantie d'exécution temps réel dur.

❑ Comment traiter les appels système Linux depuis les tâches Xenomai ? Pb?

Les appels système regroupent les accès à des ressources partagées dont la concurrence d'accès est gérée par Linux.

Or, Linux ne connaît pas les tâches Xenomai!

RTOS/Xenomai & Noyau Linux/Adaptation

- ❑ Les fameux « évènements » peuvent provoquer un rescheduling :
 - les interruptions matérielles : timer pour les tâches périodiques, équipement prêt...
 - les signaux logiciels : fin d'une tâche suite à une faute...
 - les créations/destructions de tâche : fork, exit...
 - les changements de caractéristiques d'une tâche : priorité, policy ;
 - les deschedulings volontaires.

Idée:

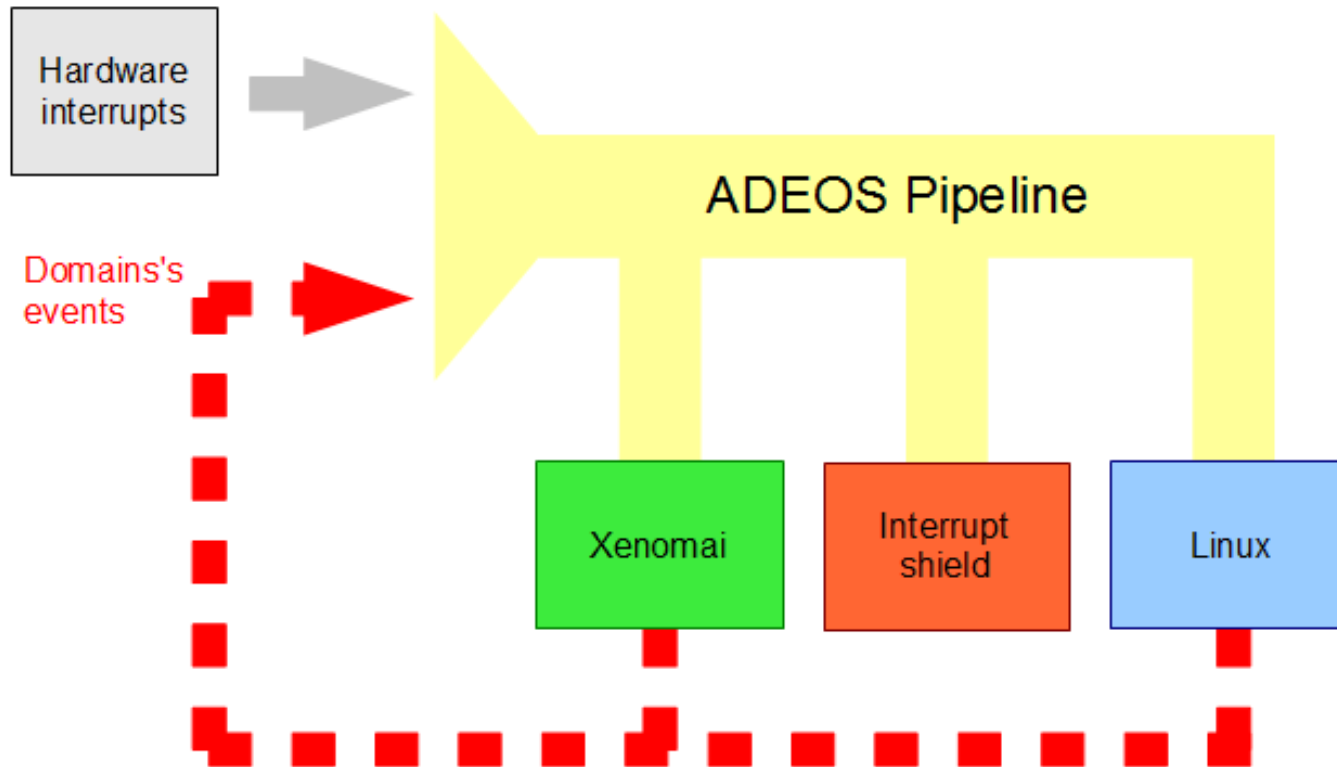
tous les évènements sont envoyés dans un tuyau et les domaines (OS) viennent y puiser les informations qui les intéressent.

RTOS/Xenomai & Linux/ADEOS

ADEOS est une couche de virtualisation de ressources matérielles permettant de faire cohabiter plusieurs noyaux sur une même machine physique.

- Repose sur l'utilisation d'un pipeline d'interruptions, nommé « I-pipe » pour « interrupt pipeline ».
- Adeos existe sous la forme d'un patch à appliquer au noyau Linux.
- I-pipe a été étendue aux différents événements pouvant se produire sur le système:
les interruptions externes (asynchrones), les exceptions (synchrones) et les appels système.
- Au sein d'ADEOS, chaque noyau est représenté par une entité appelée « domaine » qui surveille les événements circulant sur le I-pipe.
- Le domaine temps réel est servi en priorité à l'arrivée d'un événement. Le domaine racine assure le fonctionnement par défaut du système.

RTOS/Xenomai & Linux/ADEOS(i-pipe)

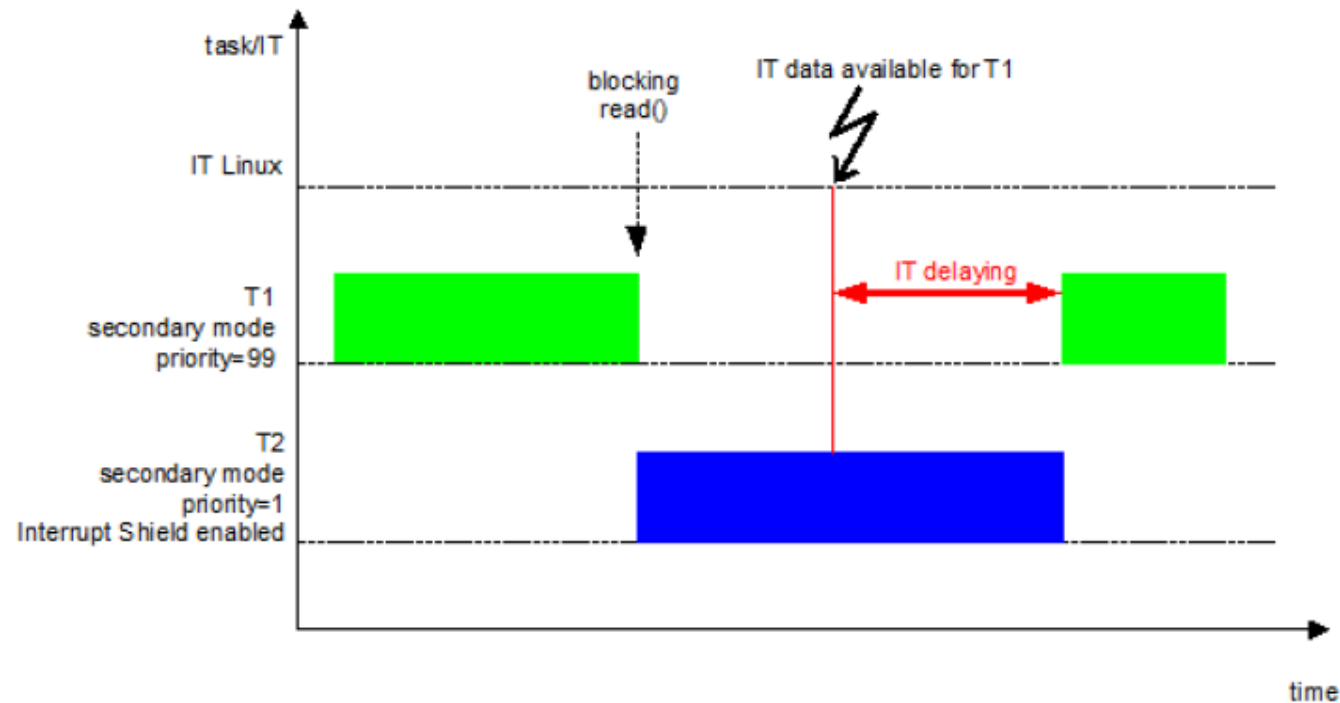


ADEOS et gestion des événements

Lors du démarrage de la machine, le domaine Linux est créé *avant* le domaine Xenomai. Cette primauté lui confère le nom de *root domain*.

RTOS/Xenomai & Linux/ADEOS(Interrupt shield)

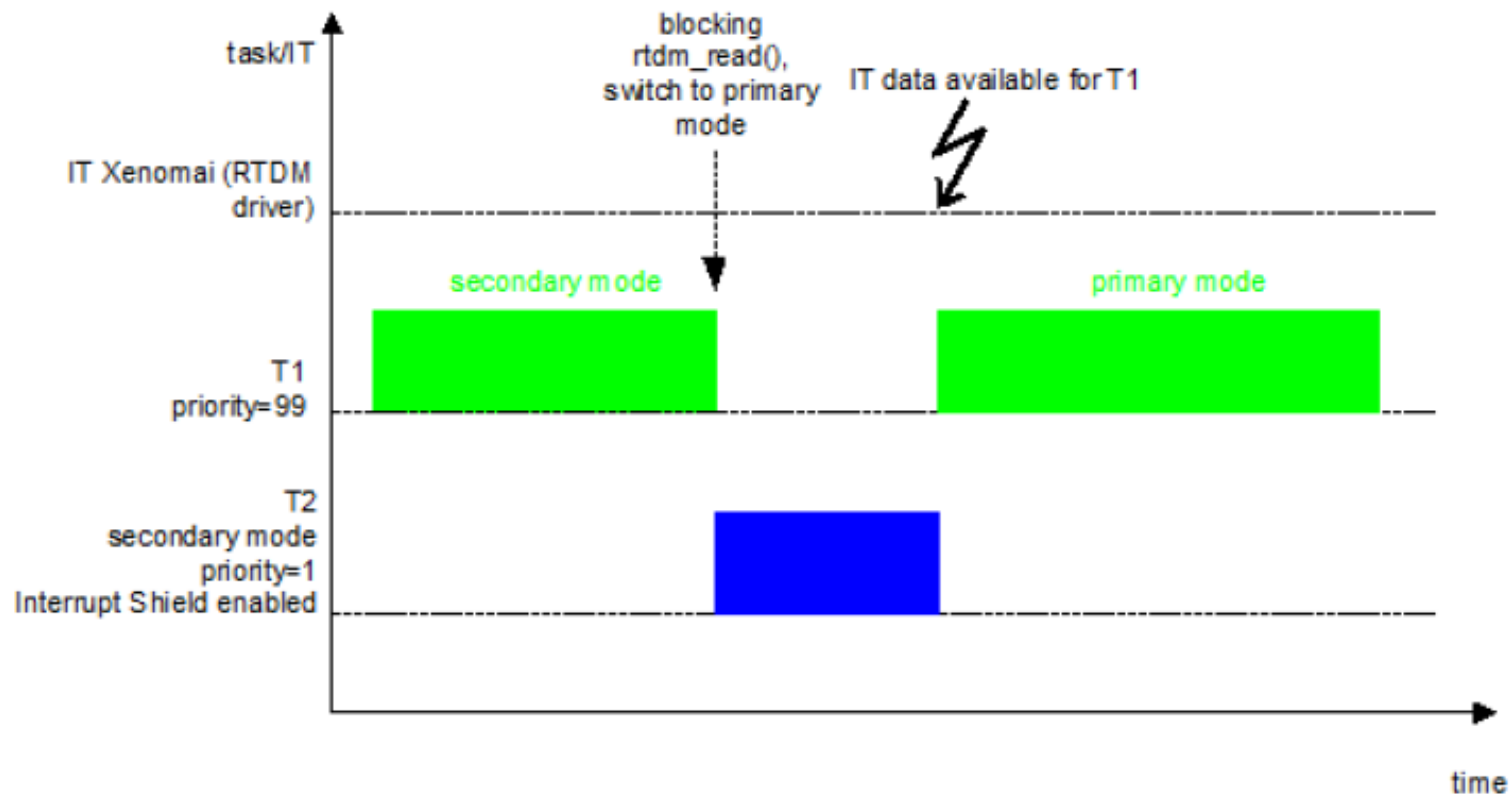
Exemple: soit deux tâches T1 et T2 en secondary mode. T2 active l'Interrupt Shield.



Une interruption arrive (IT data available for T1). T1 est censée s'exécuter, or elle est schedulée par Linux, et celui-ci n'a pas encore été informé que l'interruption est arrivée (IS). T2 termine alors son traitement, l'interruption est relâchée, T1 est débloquée. **Inversion de priorité!!**

RTOS/Xenomai & Linux/ADEOS(Interrupt shield)

Solution: RTDM (Real Time Driver Model)



Donc il faut que l'interruption pour T1 ne soit pas traitée par Linux mais directement par Xenomai.

RTOS/Xenomai & Noyau Linux/ co_scheduling

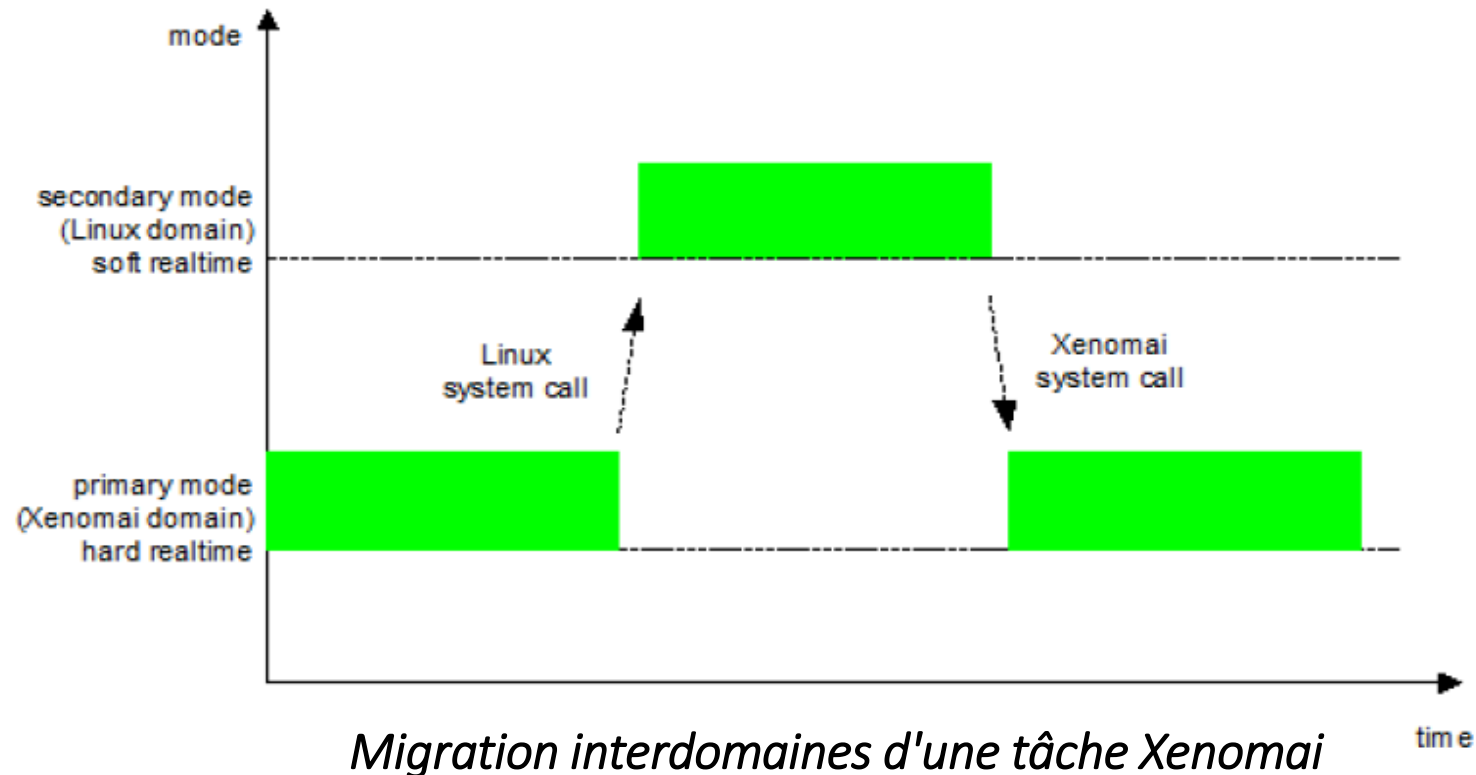
□ Solution

La solution consiste à déplacer les tâches d'un *scheduler* à l'autre en fonction des ressources utilisées (*shadow threading*).

- Une tâche schedulée par Xenomai est dite en primary mode.
- Une tâche Xenomai schedulée par Linux est dite en secondary mode .
- Une tâche Linux schedulée par Xenomai est dite... , *ça n'existe pas*.
- Le passage d'un mode à l'autre est fonction des ressources accédées et donc des appels système Xenomai ou Linux.

RTOS/ Xenomai & Noyau Linux/ co_scheduling

Exemple



Linux devient une tâche de Xenomai. Mais quelle priorité lui affecter ?

RTOS/Xenomai & Noyau Linux/Priorités

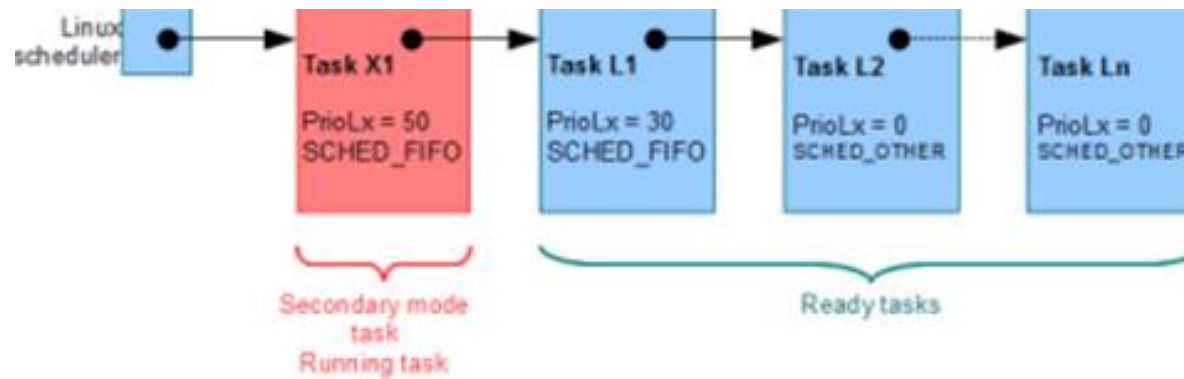
❑ La gestion des priorités dans les modes primary et secondary

- Dans l'ordonnanceur Linux, la tâche Xenomai est maintenant une tâche comme les autres → *comment faire du temps réel avec Linux ?*
- les dérives temporelles étaient introduites par :
 - ✓ les interruptions (handlersLinux) : nous avons maintenant ADEOS pour les masquer (Interrupt Shield) ;
 - ✓ la préemption par des tâches de plus forte priorité : il suffit de s'exécuter dans la classe SCHED_FIFO;
 - ✓ l'absence de préemption à l'intérieur du noyau : les options natives (« low latency desktop ») la résolvent partiellement.

RTOS/Xenomai & Noyau Linux/Priorités

□ La gestion des priorités dans les modes primary et secondary

- priorité cte dans le scheduler Linux
- Xi tâches Xenomai et Li tâches Linux
- PrioLx: priorité d'une tâche dans l'ordonnanceur de Linux
- PrioX : priorité d'une tâche dans l'ordonnanceur Xenomai.



- La priorité d'une tâche Xenomai est constante, quel que soit le scheduler dans lequel elle se trouve.

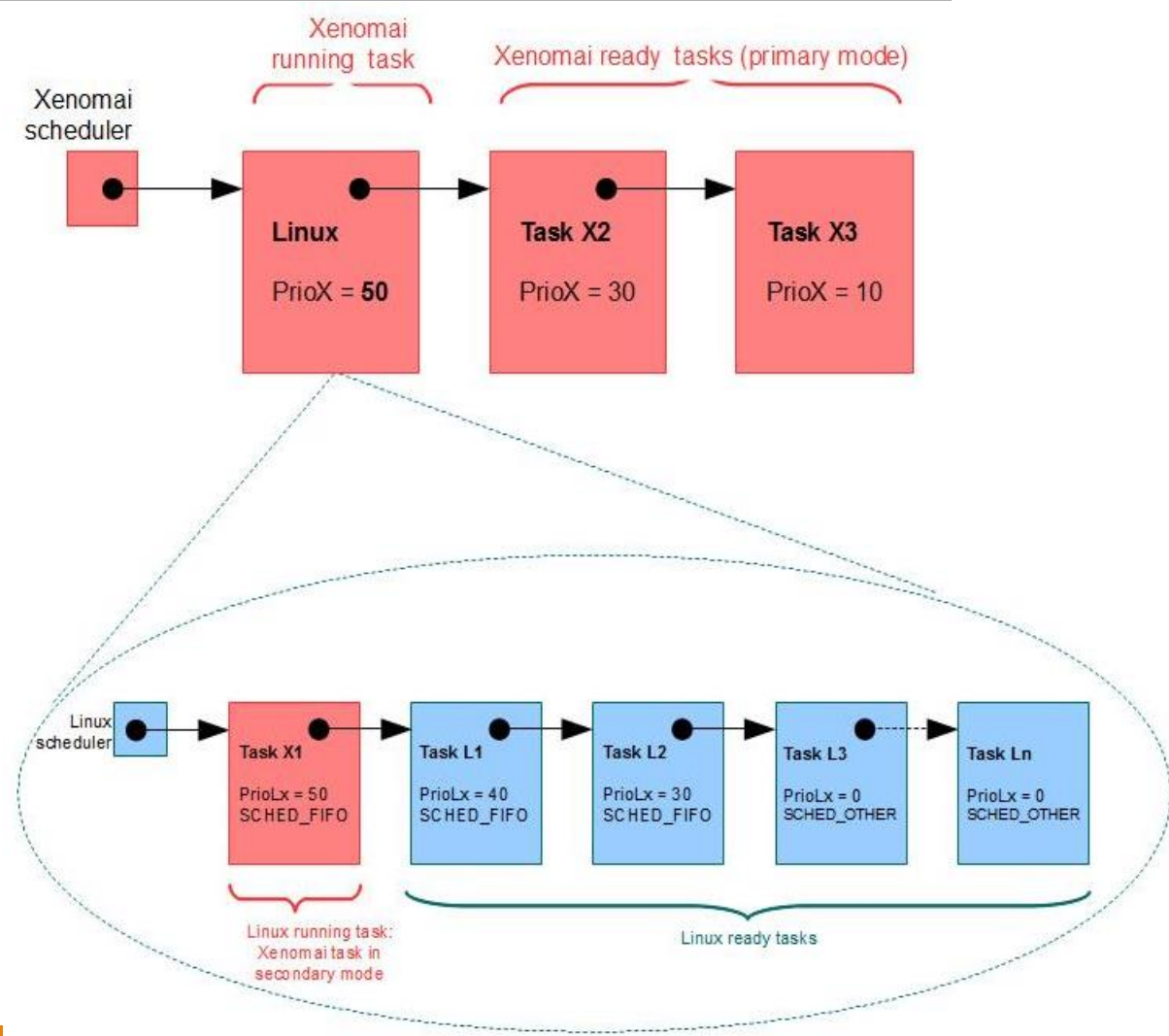
RTOS/Xenomai & Noyau Linux/co_scheduling

- ❑ **La gestion des priorités dans les modes: primaire et secondaire**
 - La priorité d'une tâche Xenomai est constante.
 - Cette implémentation n'est possible qu'à une condition : que la priorité de la tâche Linux dans l'ordonnanceur Xenomai évolue dynamiquement, sinon on se retrouverait dans un cas d'inversion de priorité.

RTOS/Xenomai & Noyau Linux/Priorités

La gestion des priorités

➤ Exemple 2: Le noyau Linux est une tâche de fond dont la priorité évolue dynamiquement en fonction de la tâche Xenomai de plus haute priorité éligible en mode secondaire.



RTOS/Xenomai & Noyau Linux/Priorités

❑ La gestion des priorités dans les modes: primaire et secondaire

➤ *Une tâche en secondary mode est du temps réel mou et son temps d'exécution est borné (Interrupt Shield).*

➤ *Pour les tâches à contrainte temps réel dur, il faut absolument rester en primary mode*

Les tâches Xenomai ont 99 niveaux de priorité possibles, exactement comme la classe SCHED_FIFO/RR.

➤ *Dans le scheduler Xenomai, la priorité de la tâche Linux (noyau) est égale :*

- *à celle de la tâche de plus haute priorité en secondary mode;*
- *à -1 s'il n'y en a pas.*

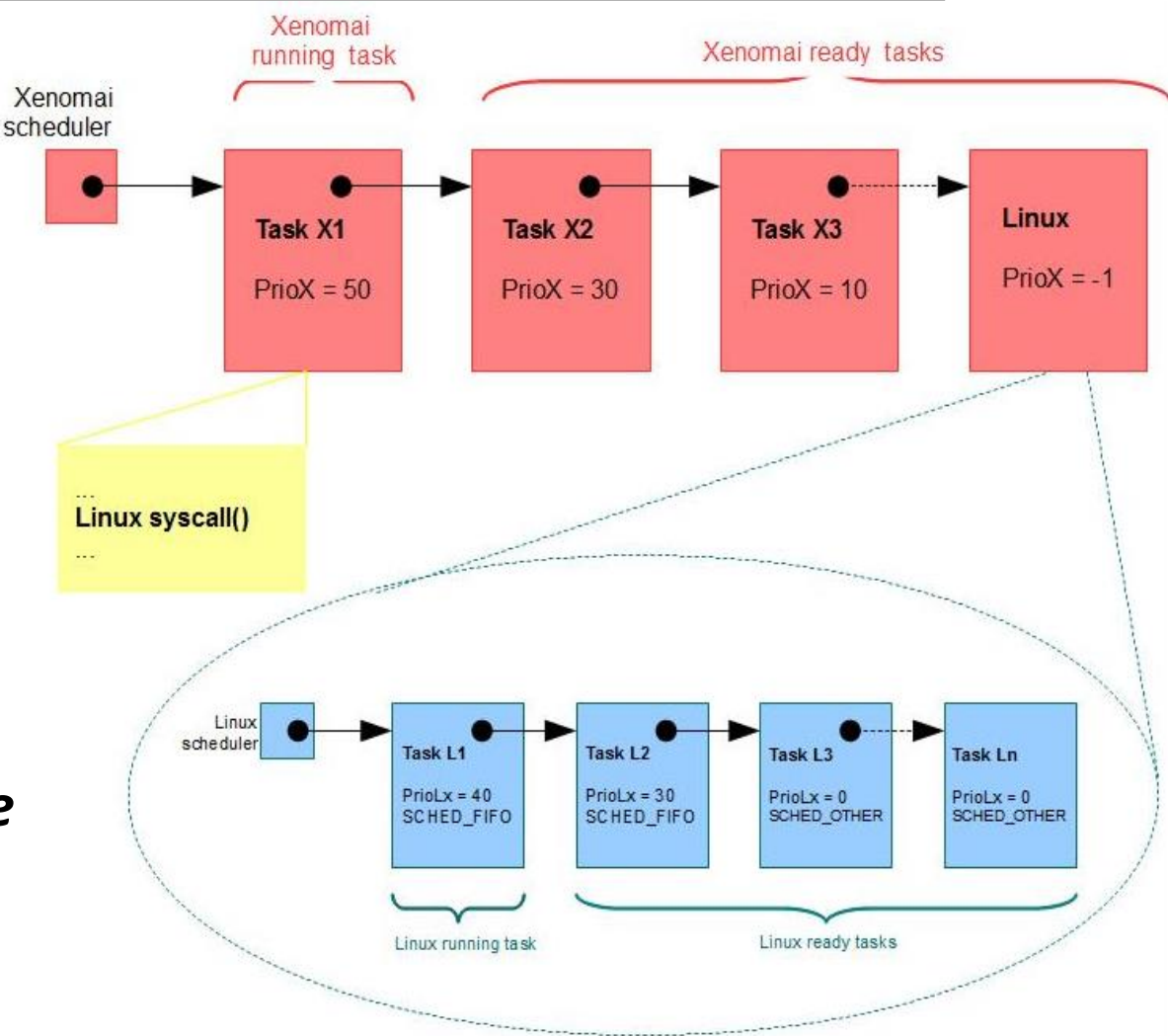
Attention : une tâche Linux peut préempter une tâche Xenomai.

RTOS/Xenomai & Noyau Linux/Priorités

La gestion des priorités

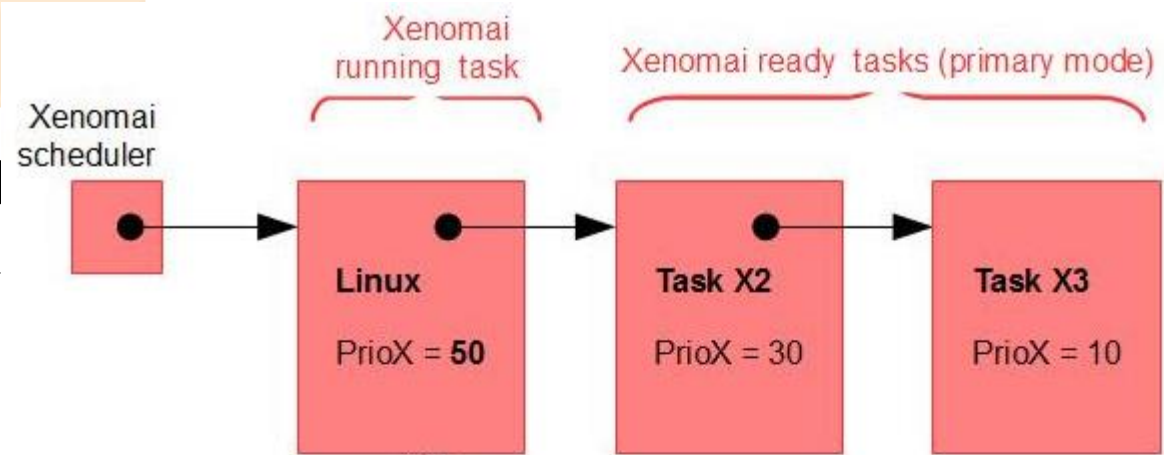
- Exemple 3: Le noyau Linux est une tâche de fond dont la priorité évolue dynamiquement en fonction de la tâche Xenomai de plus haute priorité éligible en mode secondaire.

Dans le scheduler Xenomai, la priorité de la tâche Linux est égale -1



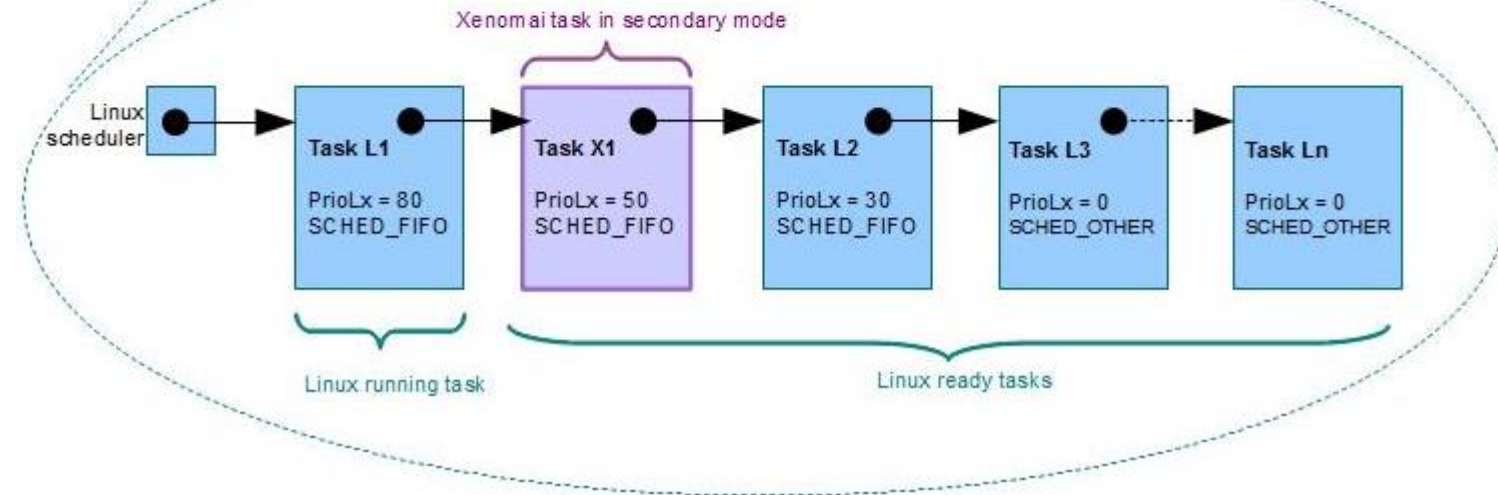
Systèmes Temps Réel & Xenomai

RTOS/Xenomai & No



La gestion des priorités

- Exemple 4: Les tâches Xenomai en mode secondaire sont en concurrence avec les autres tâches Linux de la classe SCHED_FIFO/RR..



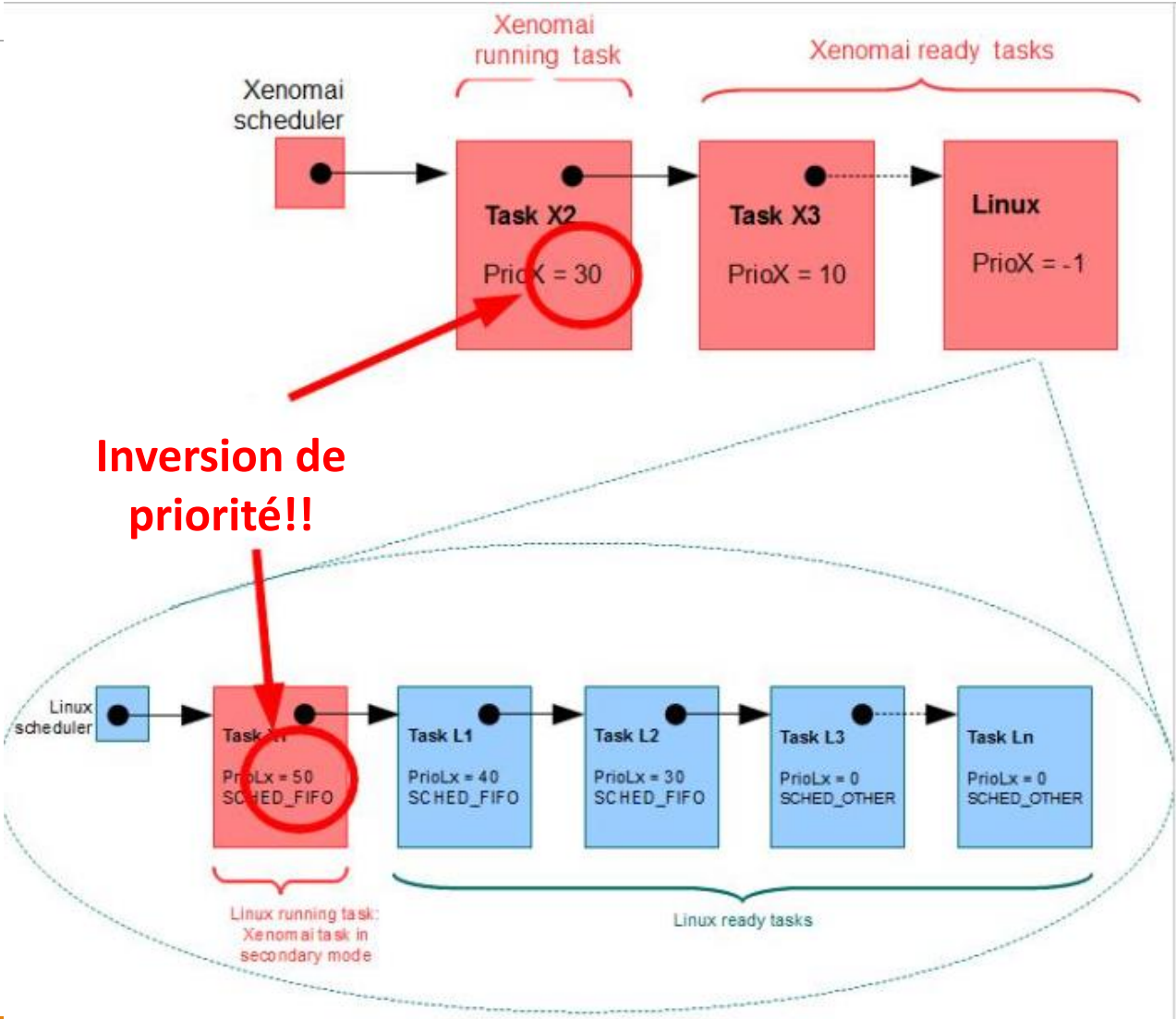
- *la priorité de la tâche Linux est égale : à celle de la tâche de plus haute priorité en secondary mode;*

RTOS/Xenomai & Noyau Linux/Priorités

La gestion des priorités

Exercice

discuter l'ordonnancement donné par le schéma ci-contre



RTOS/Xenomai & Noyau Linux/Priorités

□ La gestion des priorités

une tâche Linux peut préempter une tâche Xenomai??

- Il faut être attentif à deux choses pour les tâches en *secondary mode* afin d'éviter des comportements non souhaités :
 - leurs priorités doivent être inférieures à celles en *primary mode* ;
 - leurs priorités doivent être supérieures aux tâches Linux s'exécutant dans la classe SCHED_FIFO (dépend de la distribution)

RTOS/Xenomai

❑ L'API native et les skins pour la programmation temps réel

- Xenomai fournit une API très complète avec les fonctionnalités temps réel suivantes :
 - tâches avec 99 niveaux de priorité, *round robin* optionnel...
 - files de messages ;
 - allocation dynamique de mémoire spécifique RT ;
 - sémaphores ;
 - *watchdogs* ;
 - timers ;
 - *mutexes* ;
 - ...
- Cette API, dite « native », est complétée par des skins. Ce sont des API d'autres RTOS (pSos, VxWorks...) qui encapsulent des appels natifs.
- Les skins sont destinés au portage d'applications existantes ou à du prototypage.

RTOS/Xenomai

- ❑ L'API native et les skins pour la programmation temps réel
- Nous allons décrire les principales primitives de l'API Alchemy.

<https://doc.xenomai.org/v3/html/xeno3prm/index.html>

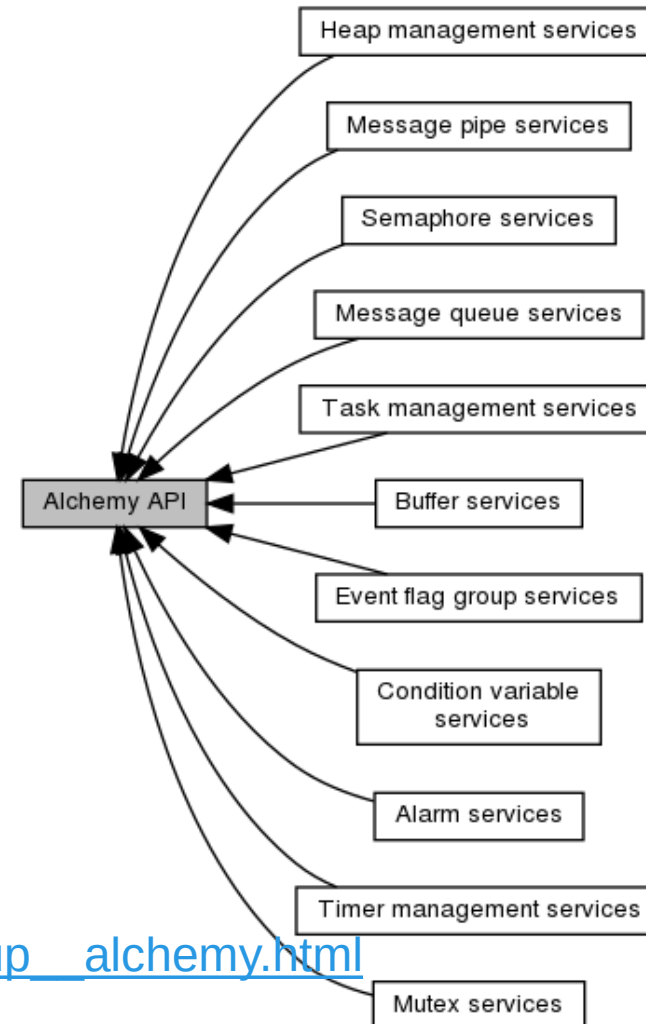
Xenomai 3.1

Main Page	Related Pages	Modules
▼ Xenomai		
API service tags		
Deprecated List		
▼ Modules		
▶ RTDM		
▶ Cobalt		
Smokey API		
▶ Alchemy API		
VxWorks® emulator		
pSOS® emulator		

RTOS/Xenomai

□ L'API native et les skins pour la programmation temps réel

➤ les principales primitives de l'API Alchemy.



https://xenomai.org/documentation/xenomai-3/html/xeno3prm/group__alchemy.html

Environnement de développement de Xenomai

- Évolutions permanentes (Xenomai 2, Xenomai 3, Xenomai 4)
- Les processus Linux peuvent utiliser l'API Xenomai
- Performances moins bonnes mais acceptables pour la plupart des applications
- API native mais supporte d'autres API (les skins) : Alchemy, POSIX, VxWorks, pSOS+, VRTX, RTAI
- Bibliothèque RTDM (Real Time Driver Model) pour l'écriture de pilotes de périphériques temps réel

Programmation RT/ Xenomai avec l'API native

- ❑ Inclure les bibliothèques et les définitions

```
#include <linux/module.h>
MODULE_LICENSE("GPL");
#include <rtdk.h>
#include <alchemy/task.h>
#include <alchemy/timer.h>
#include ...
#define HIGHEST_PRIO 99
#define PERIOD 1000000000
#define ...
```

Programmation RT/ Xenomai avec l'API native

- ❑ Code exécuté par les différentes tâches (exemple)

```
#include <linux/module.h>
```

```
RT_TASK task; //Déclarations identificateurs de tâches, sémaphores, etc.
```

```
void code_test(void *arg){
```

```
static int loop = N_LOOP;
```

```
while(loop--){
```

```
rt_printf("Hello World ! nn");}}
```

```
struct RT_TASK {  
    uintptr_t handle;  
    pthread_t thread;  
};
```

Programmation RT/ Xenomai avec l'API native

❑ Création d'une tâche

```
int rt_task_create ( RT_TASK * task,  
                    const char * name,  
                    int stksize,  
                    int prio,  
                    int mode)
```

- task est un pointeur sur une variable de type **RT_TASK**
- name : identifiant unique pour la tâche
- stksize : permet de définir la taille de la pile (0 = taille par défaut)
- prio : priorité de la tâche
- mode : mode lancement (passez 0 ou **T_JOINABLE** si nécessaire)

Programmation RT/ Xenomai avec l'API native

❑ Lancement d'une tâche

```
int rt_task_start ( RT_TASK *          task,  
                   void(*)(void *arg) entry,  
                   void *             arg)
```

- **task** est un pointeur sur une variable de type **RT_TASK**
- **entry** correspond à la fonction à exécuter par la tâche créée et qui devra avoir le prototype suivant:

```
void uneFonction(void *arg);
```

- **arg** est un pointeur passé à la fonction

Programmation RT/ Xenomai avec l'API native

❑ Création d'une tâche : Exemple

```
#include <native/task.h>
#include <native/timer.h>
void code_test(void *arg){
    static int loop = N_LOOP;
    while(loop--){
        rt_printf("Hello World ! nn");}}
int main(int argc, char *argv[]) {
    RT_TASK task;
    if ((error = rt_task_create(&task, "my first task", 0, 99, T_JOINABLE))
        != 0) {
        fprintf(stderr, "Error: cannot create the task (%s)\n", strerror(-error));
        exit(EXIT_FAILURE);}
    if ((error = rt_task_start(&task, Code_test, NULL)) != 0) {
        fprintf(stderr, "Error: cannot start the task (%s)\n", strerror(-error));
        exit(EXIT_FAILURE);}
    rt_task_join(&task);
    return EXIT_SUCCESS;}
```

Programmation RT/ Xenomai avec l'API native

❑ Création et lancement d'une tâche

```
Int rt_task_spawn( &task,           // pointeur sur la tâche
                  « My first task",   // nom de la tâche
                  STACK_SIZE,        // taille de la pile d'exécution
                  PRIORITE,           // 99 est la plus haute priorité, 0 la plus
basse
                  mode,               // mode de création de la tâche
(options)
                  &code_test,        // pointeur de la fonction à exécuter
                  args               // arguments à la fonction
```

- Regroupe la fonctionnalité des deux fonctions précédentes (rt_task_create et rt_task_start)

Programmation RT/ Xenomai avec l'API native

Fonction de création et lancement du processus temps reel dans un module

```
int init_module(void){  
    ...  
    rt_task_spawn(&task, "My first task", 0, HIGHEST_PRIO, T_FPU|T_CPU(0),  
                  &code_test, NULL);  
    ...  
    return 0;  
}
```

- Regroupe la fonctionnalité des deux fonctions précédentes (rt_task_create et rt_task_start)

Programmation RT/ Xenomai avec l'API native

❑ Mode secondaire et mode Primaire

- La fonction `printf()` fait passer une tâche temps réel dans le mode non temps réel
- La bibliothèque `rtdk` offre une version temps réel:

`rt_printf(...)`

Les arguments sont les mêmes

- **`rt_print_auto_init(1)`** doit être appelé en début de programme

Lance une tâche non temps réel responsable de gérer l'affichage

Programmation RT/ Xenomai avec l'API native

❏ Exemple 1/3

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/mman.h>
#include <alchemy/task.h>
#include <alchemy/timer.h>
#include <alchemy/sem.h>
#include <rtdk.h>
#define ITER 10

static RT_TASK t1;
static RT_TASK t2;
int global = 0;
```

Programmation RT/ Xenomai avec l'API native

□ Exemple2/3

```
void taskOne(void *arg) {
    int i;
    for (i=0; i < ITER; i++)
    {
        rt_printf("I am taskOne and global = %d.....\n", ++global);
    }
}
void taskTwo(void *arg) {
    int i;
    for (i=0; i < ITER; i++)
    {
        rt_printf("I am taskTwo and global = %d-----\n", --global);
    }
}
```

Programmation RT/ Xenomai avec l'API native

❏ Exemple 3/3

```
int main(int argc, char* argv[])
{
    /* Perform auto-init of rt_print buffers */
    rt_print_auto_init(1);

    /* Avoids memory swapping for this program */
    mlockall(MCL_CURRENT|MCL_FUTURE);
    /* create the two tasks */
    rt_task_create(&t1, "task1", 0, 1, 0);
    rt_task_create(&t2, "task2", 0, 1, 0);

    /* start the two tasks */
    rt_task_start(&t1, &taskOne, 0);
    rt_task_start(&t2, &taskTwo, 0);
    return 0;}

```


Les modules de Xenomai

- ❑ Les modules de l'API native de Xenomai :
 - **task** : gestion des tâches
 - **timer** : gestion du temps
 - **sem** : gestion des sémaphores
 - **mutex** : gestion des exclusions mutuelles
 - **queue** : gestion des boîtes aux lettres
 - **pipe** : gestion des messages par FIFO
 - **intr** : gestion des interruptions
 - etc.
- ❑ Autres modules utiles :
 - **rtchk** : gestion des affichages (rt_printf, etc.)
 - etc.

▼ Alchemy API

- ▶ Alarm services
- ▶ Buffer services
- ▶ Condition variable services
- ▶ Event flag group services
- ▶ Heap management services
- ▶ Mutex services
- ▶ Message pipe services
- ▶ Message queue services
- ▶ Semaphore services
- ▶ Task management services
- ▶ Timer management services

Programmation RT/ Xenomai avec l'API native

- ❑ Dans la fonction `init_module` ou dans la tâche directement

```
rt_task_set_periodic(&task, // pointeur sur la tâche  
                      start,   // date de démarrage  
                      periode // période de la tâche  
                      );
```

- ❑ Dans la tâche

```
rt_task_wait_period(&ov); // attendre la fin de la période
```

- ❑ Changer la priorité d'une tâche

```
rt_task_set_priority(&task, priorite);
```

Rendre une tâche périodique

□ exemple

```
void periodicTask(void *arg) {  
    RTIME nSec = ...;  
    //Set the current task as a periodic task, starting now and every nSec  
    rt_task_set_periodic(rt_task_self(), TM_NOW, nSec);  
    while ((error = rt_task_wait_period(NULL)) == 0) {  
        // Do something  
    }  
}
```

Suspendre, reprendre, terminer une tâche

❑ Suspension

```
rt_task_suspend(&task);
```

❑ Attente de la fin d'une tâche

```
rt_task_join(&task);
```

❑ Reprise ou déclenchement d'une tâche non périodique

```
rt_task_resume(&task);
```

❑ Terminaison

```
rt_task_delete(&task);
```

Gérer le temps

❑ Gestion du temps/API Alchemy

Xenomai 3.0.5

Main Page	Related Pages	Modules	Data Structures ▼
			▶ Condition variable services ▶ Event flag group services ▶ Heap management services ▶ Mutex services ▶ Message pipe services ▶ Message queue services ▶ Semaphore services ▶ Task management services ▼ Timer management services ▼ rt_timer_info
			date period RT_TIMER_INFO rt_timer_inquire rt_timer_ns2ticks rt_timer_read rt_timer_spin rt_timer_ticks2ns

Gérer le temps

❑ Récupération du temps

```
rt_timer_read(); // en "ticks" (= ns si TM_ONESHOT)
```

❑ Conversion

```
SRTIME tns, ttick;  
ttick = rt_timer_ns2tick(tns);  
tns = rt_timer_tick2ns(ttick);
```

Gérer le temps

❑ Endormir la tâche

```
rt_task_sleep(duree); // en "ticks"  
rt_task_sleep_until(date); // en "ticks"  
rt_timer_spin(duree); // chargé CPU en ns
```

Communication entre tâches : les sémaphores

- Synchronisation, précedence, etc.

```
RT_SEM sem_test;  
rt_sem_create(&sem_test,    // pointeur sur le sémaphore  
              "SEMTEST",    // nom du sémaphore  
              val,           // valeur initiale >= 0  
              mode           // options  
              );
```

- Prendre le sémaphore (attente

```
rt_sem_p(&sem_test, TM_INFINITE);
```

- Restituer le sémaphore

```
rt_sem_v(&sem_test);
```


Communication entre tâches : les sémaphores

❑ Destruction

```
rt_sem_delete(&sem_test);
```

❑ Variantes asynchrones

```
rt_sem_p(&sem_test, TM_NONBLOCK); // non bloquant si sémaphore absent  
rt_sem_p(&sem_test, duree);        // blocage pendant duree (en "ticks")  
rt_sem_p_until(&sem_test, date);    // blocage jusqu'à date (en "ticks")
```

Communication entre tâches : les FIFOs temps réel

- ❑ Communication entre tâches Xenomai et processus Linux

```
RT_PIPE pipe_test;  
rt_pipe_create(&pipe_test,           // pointeur sur la FIFO  
               "PIPETEST",           // nom de la FIFO  
               numero,               // numero de la FIFO  
               taille                // taille de la FIFO  
               );
```

Communication entre tâches : les FIFOs temps réel

- ❑ Écrire des données dans la FIFO

```
rt_pipe_write(&pipe_test, &bloc, taille, mode);
```

- ❑ Lire des données dans la FIFO (bloquant)

```
rt_pipe_read(&pipe_test, &bloc, taille, TM_INFINITE);
```

- ❑ Destruction

```
rt_pipe_delete(&pipe_test);
```

Communication entre tâches : Interruptions

❑ Fonctions associées

```
rt_intr_enable(&intr_test);           // activer l'interruption  
rt_intr_wait(&intr_test, TM_INFINITE); // espace utilisateur seulement  
rt_intr_delete(&intr_test);           // destruction de l'interruption
```

❑ Construction (espace utilisateur)

```
rt_intr_create(&intr_test, "INTRTEST", IT_CLAVIER, mode);
```

Communication entre tâches : Interruptions

- ❑ Permet des réactions rapides (traitement urgent, réveil tâche endormie)
- ❑ Construction (espace noyau)

```
#define IT_CLAVIER 1  
RT_INTR intr_test;  
void isr_test(void *args);  
rt_intr_create(&intr_test, "INTRTEST", IT_CLAVIER, &isr_test, NULL, mode);
```

- ❑ Construction (espace utilisateur)

```
rt_intr_create(&intr_test, "INTRTEST", IT_CLAVIER, mode);
```

Communication avec le matériel

- ❑ Pilote de périphérique
 - lecture des fichiers /dev/nom (sdb1, rtp0, ...)
 - utiliser les fonctions open, close, read, write
- ❑ Accès direct aux ports E/S

```
#include <sys/io.h>  
outb(valeur, adresse);  
valeur = inb(adresse);
```

Communication avec le matériel

- ❑ Pilote de périphérique
 - lecture des fichiers /dev/nom (sdb1, rtp0, ...)
 - utiliser les fonctions open, close, read, write
- ❑ Accès direct aux ports E/S

```
#include <sys/io.h>
outb(valeur, adresse);
valeur = inb(adresse);
```

TCB d'un thread

Champ	Description
Thread_id	Identificateur du thread
Run_count	Nombre de fois que le thread a été exécuté
Stack_ptr	Pointeur courant vers la pile du thread
Stack_start	Pointeur vers le début de la pile
Stack_end	Pointeur vers la fin de la pile
Stack_size	Taille réservée de la pile
Time_slice	Quantum de temps courant pour l'exécution
New_time_slice	Nouveau quantum de temps alloué à ce thread
Ready_next	Pointeur vers le prochain thread prêt
Ready_previous	Pointeur vers le thread précédent prêt
Thread_name	Nom du thread
Thread_priority	Priorité du thread
Thread_state	Etat du thread
Thread_entry	Point d'entrée (fonction) du thread
Thread_args	Arguments transmis au thread
Thread_timer	Descripteur de timer associé